Sesión 1: Contenerización con Docker

De 0 a entorno local reproducible

Objetivo: estandarizar el desarrollo y despliegue de aplicaciones

Aislamiento vs. Virtualización Tradicional

Virtualización (VM)

- SO completo por instancia
- Arranque más lento
- Mayor consumo de RAM/CPU

Contenedores (Docker)

- Kernel compartido
- Inicio en segundos
- Portables y ligeros

Idea clave: Docker empaqueta aplicación + dependencias, no un sistema operativo completo.

Caso Real: COMPÁS VIRTUAL se cae

⚠️ Crisis sin Docker

- Página caída 🔴
- SSH a servidor
- Revisar logs, configs
- ¿Qué versión de Node?
- ¿Qué commit del código?
- 30 min sin servicio 📉

✅ Respuesta con Docker

- Página caída 🔴
- `docker run compas-virtual`
- ✅ Instancia 2 levantada
- Servicio restaurado (5s)
- O replicar en 3 servidores
- Load balancer distribuye

La imagen es tu backup vivo: instancia múltiples contenedores cuando sea necesario.

Imagen vs. Contenedor

🖼️ Imagen

- **Plantilla** estática inmutable
- Capas comprimidas en un archivo
- Se construye una vez
- `docker build`
- Se descarga de Registry

📦 Contenedor

- **Instancia** ejecutable viva
- Copia de lectura-escritura de la imagen
- Múltiples contenedores de la misma imagen
- `docker run`
- Procesos en ejecución

Analógía: Imagen = **Clase** / Contenedor = **Instancia** de esa clase

Una imagen puede generar **N contenedores simultáneos**, cada uno con su estado aislado.

Arquitectura de Docker

CLI (`docker ...`) -> Docker Engine API -> Docker Daemon -> Containers / Images / Volumes / Networks

Daemon <-> Registry (`Docker Hub`) para `pull` y `push`

Piensa en Docker como una **fabrica + gestor** de entornos aislados.

Dominio de la CLI y Docker Desktop

CLI minima que debes dominar

- `docker run` crea y arranca
- `docker ps` inspecciona estado
- `docker exec` entra al contenedor
- `docker logs` depura rapido

```
docker run -d --name web -p 8080:80 nginx:alpine
docker ps
docker logs web
docker exec -it web sh
```

Docker Desktop te da visibilidad visual; la CLI te da **velocidad y control**.

Dockerfile desde cero

Un Dockerfile es una receta reproducible. Cada instrucción crea una capa.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
CMD ["node", "server.js"]
```

Regla: primero dependencias, luego codigo. Asi aprovechas cache y compila mas rapido.

Buenas Prácticas en Dockerfiles

Evitar

- `FROM node:latest`
- Copiar todo sin `.dockerignore`
- Ejecutar como root por defecto

Aplicar

- `FROM node:20-alpine`
- Multi-stage build
- Usuario no root + imagen minima

```
# stage final segura
RUN addgroup -S app && adduser -S app -G app
USER app
```

Docker Compose: Orquestacion local

web <-> db <-> redis (misma red interna)

```
services:
  web: { build: ., ports: ["3000:3000"], depends_on: [db] }
  db: { image: postgres:16, environment: [POSTGRES_PASSWORD=dev] }
  redis: { image: redis:7-alpine }
```

Con `docker compose up -d` levantas todo el entorno en un solo comando.

Workshop practico: Dockerizacion real

Ruta de trabajo

- 1. Build de imagen
- 2. Compose con app + DB
- 3. Volumen para persistencia
- 4. Smoke test de conectividad

```
git clone <repo>
docker compose build
docker compose up -d
docker compose ps
curl http://localhost:3000/health
```

Resultado esperado: entorno reproducible, persistente y listo para desarrollo.

Fin del taller

Lo que ya puedes hacer hoy

- Empaquetar una app en imagen Docker.
- Levantar un stack local con Compose.
- Aplicar buenas practicas desde el primer dia.